A Cache-Efficient Iterative Sparse Triangular Solver

Ids Rehorst — Master Student Applied Mathematics, TU Delft

TU Delft

Motivation

Why are sparse triangular solves important?

- **Ubiquitous:** arises in LU, preconditioners, domain decomposition.
- **Hard:** strict deps + irregular sparsity → latency-bound.
- **Our angle:** a little redundancy to keep blocks cache-hot.

What are we solving?

$$\begin{pmatrix} l_{11} & 0 & \cdots & 0 \\ l_{21} & l_{22} & \cdots & 0 \\ \vdots & \ddots & \ddots & \vdots \\ l_{n1} & l_{n2} & \cdots & l_{nn} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{pmatrix}$$

- **Goal:** solve a triangular linear system $\mathrm{Lx} = \mathrm{b}$
- **Focus:** sparse cases
- **Why it matters:** sparsity admits possible parallelization

Existing Methods

Level Scheduling

- **Approach:** Preprocess into independent levels; solve a level in parallel, then barrier.
- **Pros:** Simple; low inspector cost; portable.
- **Cons:** Few wide levels on irregular matrices.
- **Cache:** Weak spatial/temporal reuse.

Blocked/Supernodal

- **Approach:** Factorization forms supernodes; updates via BLAS-3; solves on dense triangular blocks.
- **Pros:** Strong cache reuse; vectorization-friendly.
- **Cons:** Needs large/regular blocks; irregular matrices fall back.
- **Cache:** Excellent locality with big panels; benefits fade on small/irregular blocks.

Dependency-graph / task parallelism

- **Approach:** Express SpTRSV as a DAG of fine tasks; runtime schedules under dependencies (OpenMP tasks, Kokkos).
- **Pros:** Exposes fine-grain TLP; adapts to irregular sparsity
- **Cons:** Runtime overhead; potential imbalance.
- **Cache:** Cross-task reads of x cause capacity/conflict misses.

Method

Step 1: Matrix Reordering with RACE

Build the row-dependency graph of L and level it with RACE[2]. RACE applies a distance-1 BFS to L. Sort rows/cols by level via a symmetric permutation P :

$$\tilde{L} = PLP^T = \begin{pmatrix} L_1 & & & & \\ B_2 & L_2 & & & \\ & B_3 & L_3 & & \\ & & \ddots & \ddots & \\ & & & B_k & L_k \end{pmatrix}$$

Each L_i is (sparse) lower-triangular on level i. B_i couples only to level $i-1$.

Step 2: Provisional Solve

Solve all diagonal blocks independently (ignoring sub-diagonal couplings):

$$\hat{x}_i = L_i^{-1}\tilde{b}_i, \qquad i = 1, \dots, k$$

$$\hat{x} = \begin{pmatrix} \hat{x}_1 \\ \hat{x}_2 \\ \vdots \\ \hat{x}_k \end{pmatrix}, \qquad \tilde{b} = Pb = \begin{pmatrix} \tilde{b}_1 \\ \tilde{b}_2 \\ \vdots \\ \tilde{b}_k \end{pmatrix}$$

Use single-thread triangular solves per block. Keep L_i cache-hot on the same core.

Step 3: Correction Sweep

Inject the one-block coupling and re-solve each block with a corrected RHS:

$$x_1 = \hat{x}_1,$$
$$x_i = L_i^{-1}(\tilde{b}_i - B_i x_{i-1}),$$
$$i = 2, \dots, k.$$

Only x_{i-1} is needed for block i, so the dependency chain is narrow. L_i is reused while still warm.

Task Schedule

Tasks per block $i = 1, \dots, k$:

- Step 2 Provisional Solve $T_i^{(1)}$: $\hat{x}_i \leftarrow L_i^{-1}\tilde{b}_i$
- Step 3 Correction Step $T_i^{(2)}$: $x_1 \leftarrow \hat{x}_1$, $x_i = L_i^{-1}(\tilde{b}_i - B_i x_{i-1})$

Dependency edges:

1. Reuse within a block: $T_i^{(1)} \rightarrow T_i^{(2)}$
2. Forward chain across blocks: $T_{i-1}^{(2)} \rightarrow T_i^{(1)}$

Step 4: Permute back to original ordering

$$x = P^T\tilde{x} = P^T \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_k \end{pmatrix}$$

Objectives

- *Maximize Cache Reuse*
- *Exposing parallelism in SpTRSV*
- *Portability*

References

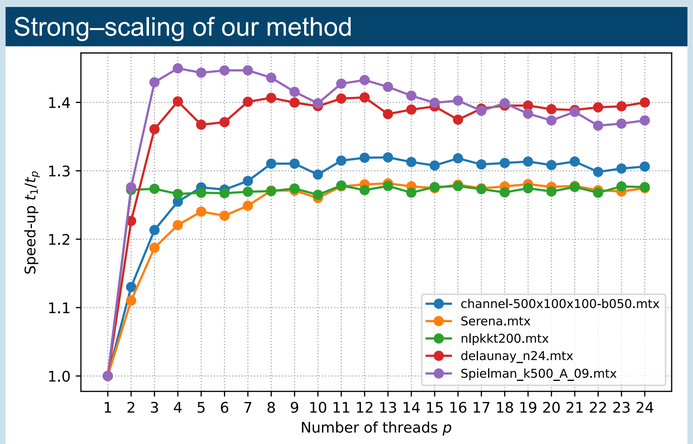
1. Rehorst, I. (2025). *A Cache-Efficient Iterative Sparse Triangular Solver*. https://repository.tudelft.nl/record/uuid:35a91086-126d-4562-b6ce-e4bcaf6c9630
2. Christie Alappat et al. "A Recursive Algebraic Coloring Technique for Hardware-efficient Sym-metric Sparse Matrix-vector Multiplication". In: ACM Trans. Parallel Comput. 7.3 (June 2020). issn:2329-4949. doi: 10.1145/3399732. url:https://doi.org/10.1145/3399732
3. Timothy A. Davis and Yifan Hu. "The university of Florida sparse matrix collection". In: ACM Trans. Math. Softw. 38.1 (Dec. 2011). issn: 0098-3500. doi: 10.1145/2049662.2049663. url: https://doi.org/10.1145/2049662.2049663.
4. OpenMP Architecture Review Board. OpenMP® Compilers & Tools.https://www.openmp.org/resources/openmp-compilers-tools/. Accessed 27 June 2025. 2025.
5. Sivasankaran Rajamanickam et al. Kokkos Kernels: Performance Portable Sparse/Dense Linear Algebra and Graph Kernels. 2021. arXiv: 2103.11991 [cs.MS]. url: https://arxiv.org/abs/2103.11991.
6. Intel® Math Kernel Library for Linux* Developer Guide. Revision 068 Intel® MKL 2020. IntelCorporation. Santa Clara, CA, 2020. url: https://cdrdv2-public.intel.com/671193/mkl-2020-developer-guide-linux.pdf.

Performance

Platform: DelftBlue compute P1 node (Intel Xeon 6248R).
Workloads: Sparse matrices from the SuiteSparse collection [3].
Table below lists the benchmark matrices used (name / size / nnz).

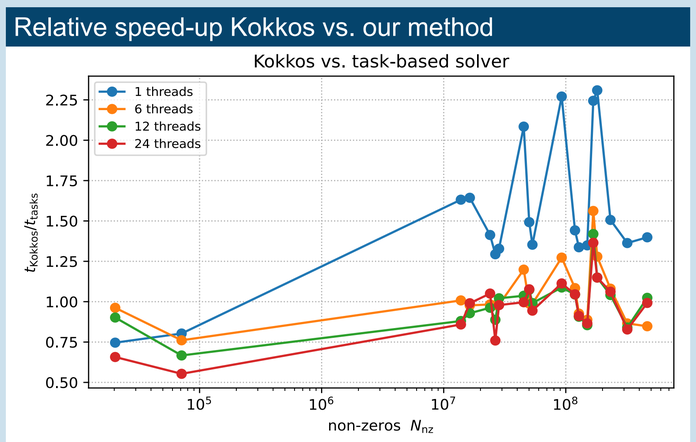
Matrix	# Rows	# Non-zeros	# Non-zeros per row	Size (MiB)
channel-500x100x100-b050.mtx	464954157	183299749	96	2116.0
Serena.mtx	1391349	319059987	229	3656.7
nlpkkt200.mtx	16240000	232232816	14	2719.6
delaunay_n24.mtx	16777216	180920994	10	2134.5
Spielman_k500_A_09.mtx	41792002	167983965	4	2081.8

Strong–scaling of our method



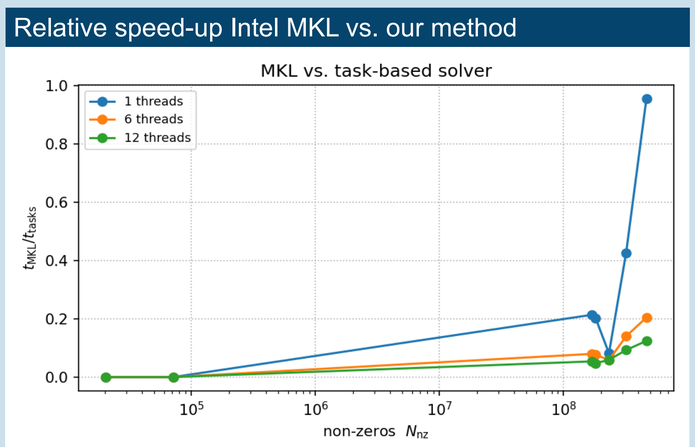
Strong scaling: ~1.25–1.4× from 1→10 threads.
Knee: most gains by 5–6 threads; then the curve flattens.
Bottlenecks: sequential correction sweep + memory bandwidth limits.
Structure effect: wider levels (more concurrent blocks) → ~1.4×; thin pipelines → ~1.25×.

Relative speed-up Kokkos vs. our method



Baseline: Compare to Kokkos SpTRSV: portable, task/level-based, widely used across architectures.
Result: Our provisional+correction solver is on par for large matrices.
Advantage: Sometimes up to ~1.3× faster in multithreaded runs.
Overhead: Lower single-thread overhead than Kokkos.

Relative speed-up Intel MKL vs. our method



Baseline: Compare to Intel MKL (vendor-optimized SpTRSV).
Result: Up to ~10× faster on large matrices.
Context: Our prototype prioritizes portability and block-level scheduling, not vendor tuning.

Implementation

The method above was implemented using:

- OpenMP 5.0 tasks + affinity(...) (requires GCC 15+[4])for the parallelization.
- RACE for the reordering.
- CSR to store matrices
- Baselines: MKL and Kokkos SpTRSV.

The core of the OpenMP implementation is shown below:

```
#pragma omp parallel default(none) shared(B,stagePtr,bp,xp,k)
{
  #pragma omp single
  {
    /* ---- Phase 1 : provisional solves ---- */
    for(int i=0;i<k;++i){
      int r0 = stagePtr[i];
      int m  = stagePtr[i+1]-r0;
      #pragma omp task                                          \
              depend(out: xp[r0:m])                             \
              affinity(  xp[r0]   )                             \
              firstprivate(r0,m)
        provisional_solve(i);
    }

    /* ---- Phase 2 : correction solves ---- */
    for(int i=1;i<k;++i){
      int r0  = stagePtr[i]       ,  m  = stagePtr[i+1]-r0;
      int r00 = stagePtr[i-1]     ,  m0 = stagePtr[i]-r00;
      #pragma omp task                                          \
              depend(in:    xp[r00:m0])                         \
              depend(inout: xp[r0 :m ])                         \
              affinity(  xp[r0]   )                             \
              firstprivate(r0,m,r00,m0)
        correction_solve(i);
    }
    /* implicit taskwait */
  }
}
```

Takeaways

- **Performance:** On par with Kokkos for large matrices; MKL remains much faster.
- **Portability:** High (method itself not depend on platform).
- **Status:** Simple, cache-aware prototype with minimal low-level tuning.
- **Outcome:** Correct results, modest scaling, easy to implement

Want to know more?

Scan the QR-code to access the GitHub page of this project. The thesis about this project[1] can also be found there:

You can contact me at:
I.W.D.H.Rehorst@student.tudelft.nl